

```
// ABSOLUTE BEGINNER · WINDOWS
```

JavaScript Fundamentals

The core building blocks of every JS program — variables, types, functions, loops, objects, and arrays. Edit any example below and run it live.

● ● ● playground.js

▶ Run

```
// Edit this and press Run – it executes right here in the browser.

const greeting = "Hello, JavaScript!";
const year = new Date().getFullYear();

console.log(greeting);
console.log("It is currently", year);
```

OUTPUT

Press Run to see output...

// every code block in this guide is editable and runnable

Variables

Data types

Operators

Conditionals

Loops

Functions

Arrays

Objects

Errors

VARIABLES

Variables – storing values

A variable is a named container for a value. JavaScript gives you three keywords to create them: `const`, `let`, and `var`. As a beginner, use `const` by default and `let` when you need to change the value later. Ignore `var` — it's older and has confusing rules.

```
// const – value cannot be reassigned
const name = "Ada";
const age = 30;
```

```
// let - value can change
let score = 0;
score = score + 10;    // this is fine with let

// This would throw an error - const cannot be reassigned:
// name = "Grace";    - TypeError

console.log(name, age, score);
```

 **Rule of thumb:** start with `const` . If your editor complains that you're trying to reassign it, change it to `let` . This habit catches bugs early.

TRY IT

▶ Run

```
const city = "Johannesburg";
let temperature = 18;
temperature = temperature + 3;
console.log(city, "temp:" + temperature + "°C").
```

...

DATA TYPES

Data types – what values can be

Every value in JavaScript has a type. The six you'll encounter every day:

TYPE	EXAMPLE	WHAT IT IS
<code>string</code>	<code>"hello"</code>	Text — always in quotes
<code>number</code>	<code>42</code> / <code>3.14</code>	Any number, integer or decimal
<code>boolean</code>	<code>true</code> / <code>false</code>	A yes/no flag
<code>null</code>	<code>null</code>	Intentionally empty value
<code>undefined</code>	<code>undefined</code>	Variable declared but not given a value
<code>object</code>	<code>{ name: "Ada" }</code>	A collection of key-value pairs (covered later)

```
const text    = "JavaScript";    // string
const version = 2024;            // number
const isReady = true;           // boolean
let nothing;                      // undefined (no value assigned)
```

```
// typeof tells you the type of any value
console.log(typeof text);    // "string"
console.log(typeof version); // "number"
console.log(typeof isReady); // "boolean"
console.log(typeof nothing); // "undefined"
```

Template literals let you embed variables inside strings using backticks instead of quotes:

```
const item = "coffee";
const price = 4.5;

// Old way - messy
console.log("A " + item + " costs $" + price);

// Template literal - clean
console.log(`A ${item} costs ${price}`);
```

TRY IT

▶ Run

```
const product = "keyboard";
const units = 3;
const unitPrice = 49.99;
const total = units * unitPrice;
```

...

OPERATORS

Operators – doing things with values

Arithmetic operators work like you'd expect from maths:

```
// + - * / % **
console.log(10 + 3); // 13 - addition
console.log(10 - 3); // 7 - subtraction
console.log(10 * 3); // 30 - multiplication
console.log(10 / 3); // 3.33... - division
console.log(10 % 3); // 1 - remainder (modulo)
console.log(2 ** 8); // 256 - exponentiation
```

Comparison operators return `true` or `false` :

```
// Always use === (strict equality), not == (loose equality)
console.log(5 === 5); // true
console.log(5 === "5"); // false - different types!
console.log(5 !== 3); // true - not equal
```

```
console.log(10 > 3);    // true
console.log(10 >= 10);  // true
```

⚠ **Always use `===`, never `==`**. The double-equals `==` tries to convert types before comparing, which leads to surprising results like `0 == false` being `true`. Triple-equals is precise and predictable.

Logical operators combine booleans:

```
const isAdult = true;
const hasTicket = false;

console.log(isAdult && hasTicket); // false - AND: both must be true
console.log(isAdult || hasTicket); // true  - OR: at least one must be true
console.log(!isAdult);             // false - NOT: flips the value
```

TRY IT

▶ Run

```
const a = 15;
const b = 4;
console.log(`${a} % ${b} =`, a % b);           // remainder
console.log(`${a} > 10 AND even?` , a > 10 && a % 2 === 0);
```

...

CONDITIONALS

Conditionals – making decisions

An `if` statement runs a block of code only when a condition is `true`.

```
const temperature = 28;

if (temperature > 30) {
  console.log("It's hot – stay hydrated.");
} else if (temperature > 20) {
  console.log("Comfortable weather.");
} else {
  console.log("It's cold – grab a jacket.");
}
```

The **ternary operator** is a compact one-liner `if/else` for simple cases:

```
const age      = 20;
const status = age >= 18 ? "adult" : "minor";
```

```
//      condition ? if true : if false
console.log(status); // "adult"
```

Use a `switch` when you have many specific values to check:

```
const day = "Monday";

switch (day) {
  case "Saturday":
  case "Sunday":
    console.log("Weekend!");
    break;
  default:
    console.log("Weekday.");
}
```

TRY IT — CHANGE SCORE TO SEE DIFFERENT RESULTS

▶ Run

```
const score = 73;

let grade;
if (score >= 90)    grade = "A";
...

```

LOOPS

Loops — repeating actions

A loop runs a block of code multiple times. The `for` loop is the most common:

```
// for (start; condition; step)
for (let i = 1; i <= 5; i++) {
  console.log(`Step ${i}`);
}
// prints Step 1, Step 2 ... Step 5
```

A `while` loop keeps running as long as its condition is `true` — useful when you don't know in advance how many iterations you'll need:

```
let coins = 1;

while (coins < 100) {
  coins = coins * 2;  // double each time
}
```

```
console.log(`Coins after doubling: ${coins}`);
```



Avoid infinite loops. A `while` loop that never becomes `false` will freeze your program. Always make sure something inside the loop moves toward the exit condition.

TRY IT — PRINT THE 3× TABLE

▶ Run

```
for (let i = 1; i <= 10; i++) {  
  console.log(`3 × ${i} = ${3 * i}`);  
}
```

...

FUNCTIONS

Functions — reusable blocks of code

A function is a named block of code you can call (run) whenever you need it. Instead of repeating the same logic everywhere, you write it once and call it by name.

```
// Define — write the function once  
function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
// Call — use it as many times as you like  
console.log(greet("Ada"));    // Hello, Ada!  
console.log(greet("Grace")); // Hello, Grace!
```

Arrow functions are a shorter way to write functions, especially for small single-purpose ones:

```
// Traditional  
function square(n) { return n * n; }  
  
// Arrow — same thing, shorter syntax  
const square2 = (n) => n * n;  
  
// Arrow with multiple lines needs curly braces + return  
const clamp = (value, min, max) => {  
  if (value < min) return min;  
  if (value > max) return max;  
  return value;  
};
```

```
console.log(square2(7));           // 49
console.log(clamp(150, 0, 100)); // 100
```

Default parameters let you give a parameter a fallback value:

```
function welcome(name = "stranger") {
  console.log(`Welcome, ${name}!`);
}

welcome("Lena"); // Welcome, Lena!
welcome();      // Welcome, stranger!
```

TRY IT

▶ Run

```
const celsiusToFahrenheit = (c) => (c * 9/5) + 32;

console.log(celsiusToFahrenheit(0)); // freezing
console.log(celsiusToFahrenheit(100)); // boiling
```

...

ARRAYS

Arrays – ordered lists of values

An array stores multiple values in a single variable, in a specific order. Each item has an index starting from `0`.

```
const fruits = ["apple", "banana", "cherry"];

console.log(fruits[0]); // "apple" – first item
console.log(fruits[2]); // "cherry" – third item
console.log(fruits.length); // 3

fruits.push("date"); // add to end
fruits.pop(); // remove from end
console.log(fruits); // ["apple", "banana", "cherry"]
```

The three array methods you'll use most often:

```
const numbers = [1, 2, 3, 4, 5];

// forEach – run a function for each item (no return value)
numbers.forEach(n => console.log(n * 2));

// map – transform every item, returns a new array
const doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10]
```

```
// filter - keep only items where the condition is true
const evens = numbers.filter(n => n % 2 === 0);
console.log(evens);    // [2, 4]
```

TRY IT

▶ Run

```
const prices = [12.5, 8.0, 23.99, 5.5, 17.0];

// Get only prices over $10
const expensive = prices.filter(n => n > 10);
```

...

OBJECTS

Objects – grouping related data

An object stores a collection of named values (called **properties**). Where an array uses numbered positions, an object uses named keys. Use objects when the pieces of data belong together and each has a distinct name.

```
const user = {
  name: "Ada Lovelace",
  age: 36,
  isActive: true,
};

// Access with dot notation
console.log(user.name);    // "Ada Lovelace"

// Access with bracket notation (useful for dynamic keys)
const key = "age";
console.log(user[key]);    // 36

// Add or change a property
user.email = "ada@example.com";
console.log(user.email);
```

Destructuring is a shortcut to pull properties out of an object into variables:

```
const product = { id: 42, name: "Keyboard", price: 89.99 };

// Without destructuring
const name1 = product.name;
const price1 = product.price;

// With destructuring - same result, shorter
```



```
const { name: productName, price } = product;
console.log(`${productName} costs $$${price}`);
```

Objects can also contain **methods** — functions that live inside the object:

```
const counter = {
  count: 0,
  increment() { this.count++; },
  reset()     { this.count = 0; },
};

counter.increment();
counter.increment();
console.log(counter.count); // 2
counter.reset();
console.log(counter.count); // 0
```

TRY IT

▶ Run

```
const book = {
  title: "The Pragmatic Programmer",
  author: "Hunt & Thomas",
  pages: 352
}
```

...

ERRORS — when things go wrong

Errors are not failures — they're information. JavaScript will throw an error when it can't do what you asked, and it will tell you exactly where and why. Use `try/catch` to handle errors gracefully instead of letting them crash your program.

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
}

try {
  console.log(divide(10, 2)); // 5 - works fine
  console.log(divide(10, 0)); // throws!
} catch (error) {
  console.log("Caught an error:", error.message);
} finally {
  console.log("This always runs.");
}
```

The three most common errors you'll see as a beginner:

ERROR	WHAT IT MEANS	COMMON CAUSE
<code>ReferenceError</code>	Variable doesn't exist	Typo in variable name, or used before <code>const</code> / <code>let</code>
<code>TypeError</code>	Wrong type for the operation	Calling a non-function, accessing property on <code>null</code>
<code>SyntaxError</code>	Code can't be parsed	Missing bracket, comma, or quote



Read error messages carefully. They contain the error type, a description, and the exact file and line number. Starting from the line number is almost always faster than re-reading all your code top to bottom.

TRY IT — INTENTIONALLY TRIGGER AND CATCH AN ERROR

▶ Run

```
function parseAge(input) {  
  const n = Number(input);  
  if (isNaN(n)) throw new Error(`"${input}" is not a valid age`);  
  if (n < 0 || n > 150) throw new Error(`Age ${n} is out of range`);  
}
```

...

// fundamentals complete

You now have the full vocabulary of the language — variables, types, operators, conditionals, loops, functions, arrays, objects, and errors. Everything else in JavaScript is built from these pieces.

[Part 03 — DOM & Browser JS](#)

[Part 04 — Async & Fetch](#)

javascript.info ↗